Agentic Knowledge Graphs over Code History for Enterprise Software Automation

Engagement Structure

University Principal Investigator, Professor and Researcher(s)	Dr. Chris Brown (PI, Virginia Tech), Jason Cusati (GRA, Virginia Tech)
University	Virginia Tech, USA
eBay Co-Investigator(s)	Ramesh Periyathambi
eBay VP/Head of Department	*[To be confirmed with Ramesh Periyathambi]*
Grant Amount Requested	$120,000 (one-year grant)

Objectives

At eBay, adding a new signal to a ViewItem page or wiring up an A/B-test variant takes developers 2–4 weeks, every time: the files differ, the APIs have shifted, and the team conventions are undocumented, so someone rediscovers all of it from scratch. Across a codebase of this scale the pattern repeats hundreds of times a year. The cost is not hypothetical—code-change tasks account for roughly 70% of development expenditure [15], and enterprise deployments of AI developer tools report substantial reductions in review-cycle time and onboarding load once that historical context is made available [13, 14].

Our central idea is a *substrate*: an **agentic knowledge graph** (KG) mined from eBay's own version-control history—git commits, pull requests, and code-review discussions—rather than from a static snapshot of current code. This history-grounded KG serves one purpose twice over. It is the retrieval substrate an AI agent uses to *generate* a recurring change consistent with past practice, and the reference an automated gate uses to *validate* the resulting pull request against the team's own historical conventions. Because every recommendation traces to specific historical artifacts, generations are auditable and reproducible from a fixed KG state.

Why academic research? Mining actionable patterns from a production codebase's *history*, representing them in a queryable provenance-aware KG, and using that single substrate for both generation and convention-validation are open problems spanning software engineering, AI, and knowledge representation. They require systematic investigation, not engineering alone, and exceed what an in-house team has the bandwidth to explore.

Previous Work

Developed with Ramesh Periyathambi at eBay, this proposal engages a fast-moving literature; we position our contribution by what it does *not* claim as much as by what it does.

KG-grounded code generation is established—we extend it. Repository-level code generation grounded in a code graph, with retrieval/generation/verification agents, is an active 2024–2025 area: GRACG [4] already proposes the same agent decomposition; SemanticForge [5] adds constraint-guided generation over a repository KG; Athale and Vaddina [6] and the SWE-agent AutoCodeRover [7] show graph- and AST-grounded agents resolving real tasks. We adopt this machinery rather than reinvent it. What these systems do not do is build the graph from version-control *history*—they encode a current-code snapshot, not the commits, PRs, and reviews that record how and why the code reached its present form.

Automated code review is maturing—we make it convention-aware. Learning review edits from history [8] and agentic PR review against company rules [9] are established; recent work argues such tools should *support* rather than replace reviewers, to preserve knowledge transfer and shared ownership [10]. Our validation gate follows that principle, checking generated PRs against conventions mined from the team's own review history rather than generic best practices.

History mining and institutional knowledge—our substrate. Frequent-subgraph mining over commit history yields interpretable, traceable change patterns [11], and LLM-augmented transformation-by-example automates recurring changes accepted into real projects [12]. For institutional knowledge, the closest work, Knowledge Activation [13], turns enterprise knowledge into a graph that agents traverse—but from *human-curated* units, and it explicitly flags staleness as unsolved. A KG mined continuously from version control is exactly the missing, self-updating substrate.

Our foundations. Our ICSE-FOSE 2026 position paper [1] argues for structured knowledge accumulation in software engineering; our DOE *Genesis* project [2] contributes the provenance-aware agentic-KG architecture (AST extraction, ranking/validation agents, provenance tracking) that transfers to eBay's Java context; VersiCode [3] shows version-aware code knowledge can be structured. eBay's enterprise KG, **Compass**, already links Jira, repos, commits, PRs, and deployments; our KG extends it with the code-level pattern knowledge tickets don't capture.

The gap. No prior system uses a provenance-aware KG mined from an organization's own code history as the **unified substrate** for both generation and convention-validation—the contribution of this proposal.

Methodologies / Approaches

We pursue two complementary strands, deliberately redundant to reduce risk.

Approach 1: A provenance-aware code-history KG. Working with Ramesh's team, we construct a KG over eBay's Java codebase that unifies two layers: structural facts from Abstract Syntax Trees (API usage, dependencies, conventions) and *historical* facts from git commits, PR threads, and code reviews. Frequent sub-graphs form a provenance-tagged pattern library—for example, the ViewItem signal pattern as entity nodes (`ApiGateway`, `SignalHandler`, `MetricsClient`) and typed edges (`imports`, `implements`, `dependsOn`), each linked back to the commits and reviews that established it.

Approach 2: Agentic orchestration over the shared substrate. Three role-specific agents use the same KG: *ranking agents* score candidate patterns by relevance; *generation agents* produce code conditioned on the ranked, provenance-tagged patterns rather than on LLM priors alone; and

a *validation gate* checks the resulting PR against KG-encoded historical conventions, surfacing deviations and breaking dependency changes for human attention—supporting reviewers [10], not replacing them.

The single substrate is the point: the same provenance-tagged history that conditions generation also defines what conformance means at review time. Because retrieval is keyed to a fixed KG state and every output carries a provenance trace, generations are *auditable and reproducible*—a reviewer can see exactly which historical patterns produced a change. (We treat strict determinism as a supporting design property rather than a headline; constraint-based approaches to deterministic generation are addressed elsewhere [5].)

Impact

If successful, this project cuts the time for recurring changes from weeks to hours and turns eBay's code history into persistent organizational memory: new hires, offshore teams, and internal transfers query conventions and dependency ownership instead of shadowing senior engineers—the same lever enterprise tools report using to reduce review-cycle time [14] and ease onboarding [13].

The validation gate directly targets eBay's review bottleneck for AI-generated PRs. Rather than manually reviewing every agentic change, reviewers see automated, convention-grounded conformance checks and focus on genuine anomalies—support, not replacement [10].

Risks. eBay's codebase is large; we de-risk by scoping to one well-bounded pattern (ViewItem signals) as a case study. KG extraction reliability is an open problem, addressed with staged evaluation and manual validation. The closest related systems target generation *accuracy* or rely on *curated* knowledge; our bet is that the organization's own version-control history is the higher-leverage, self-updating substrate.

Expected Product / Output / Milestones

Deliverables:

- **PoC Agentic KG** (Months 1–7): Scoped to ViewItem signals, with pattern mining and agentic orchestration.
- **Validation Framework** (Months 8–10): Automated conformance checking for agent-generated PRs.
- **Empirical Evaluation** (Months 11–12): Developer time reduction, pattern retrieval accuracy, PR acceptance rates.
- **Publications:** Academic papers at ICSE, FSE, MSR disseminating findings.

Metrics for success: Pattern retrieval precision/recall; agent-generated code acceptance rate; reduction in developer time per pattern instance. Tiered funding could support progressive expansion to additional pattern types across eBay's organization.

University / Lab / Researcher Qualification

Dr. Chris Brown (PI, Virginia Tech). Dr. Brown leads the Software Engineering and Digital Society lab, researching human-centered AI for software engineering and the use of historical development data to support developers. He has published at ICSE, FSE, ESEM, and CSCW, and is PI on the DOE *Genesis* PA-AKG project [2], whose provenance-aware agentic-KG architecture directly informs this proposal.

Jason Cusati (GRA, Virginia Tech). Mr. Cusati is a PhD candidate whose research focuses on provenance-aware knowledge graphs over code history and AI-assisted software engineering. He co-authored the ICSE-FOSE 2026 paper on structured knowledge accumulation [1] and is the lead researcher on the PA-AKG project [2], the architectural basis for the history-grounded substrate proposed here.

References

[1] J. Cusati and C. Brown. "A Case for Structured Knowledge Accumulation in Software Engineering Research." In *Proc. ICSE-FOSE*, 2026.

[2] C. Brown and J. Cusati. "PA-AKG: Provenance-Aware Agentic Knowledge Graphs." DOE Genesis Mission project, Virginia Tech, 2026.

[3] T. Wu et al. "VersiCode: Towards Version-controllable Code Generation." arXiv:2406.07411, 2024.

[4] K. Fedorov et al. "GRACG: Graph Retrieval Augmented Code Generation." In *IEEE/ACM ASEW*, 2025.

[5] W. Zhang et al. "SemanticForge: Repository-Level Code Generation through Semantic Knowledge Graphs and Constraint Satisfaction." arXiv, 2025.

[6] M. Athale and V. Vaddina. "Knowledge Graph Based Repository-Level Code Generation." In *IEEE/ACM Wksp. on LLMs for Code (LLM4Code)*, 2025.

[7] Y. Zhang et al. "AutoCodeRover: Autonomous Program Improvement." In *Proc. ACM ISSTA*, 2024.

[8] R. Tufano et al. "Using Pre-Trained Models to Boost Code Review Automation." In *Proc. ICSE*, 2022.

[9] T. M. N. Nimraka et al. "An Agentic-AI Solution for Intelligent Code Review." In *SLAAI-ICAI*, 2025.

[10] L. Heander et al. "Support, Not Automation: Towards AI-supported Code Review for Code Quality and Beyond." In *Proc. ACM FSE*, 2025.

[11] M. Janke and P. Mäder. "Graph Based Mining of Code Change Patterns From Version Control Commits." *IEEE Trans. Softw. Eng.*, 2022.

[12] M. Dilhara et al. "Unprecedented Code Change Automation: The Fusion of LLMs and Transformation by Example." *Proc. ACM Softw. Eng.* (FSE), 2024.

[13] G. Bakal. "Knowledge Activation: AI Skills as the Institutional Knowledge Primitive for Agentic Software Development." arXiv:2603.14805, 2026.

[14] A. Kumar et al. "Intuition to Evidence: Measuring AI's True Impact on Developer Productivity." arXiv, 2025.

[15] B. Lin et al. "CCT5: A Code-Change-Oriented Pre-trained Model." In *Proc. ACM ESEC/FSE*, 2023.